Reinforcement Learning for Combinatorial Optimization

Jeroen van Riel

October 12, 2025

First, we briefly provide a broad overview of the Markov decision process (MDP) framework, based on Chapter 2 of Puterman's classic reference work [1]. This discussion assumes that the reader is already familiar with the MDP framework and is primarily intended to establish some notation, specify some specific assumptions to show which parts of the general framework are most relevant in the context of modeling algorithms for combinatorial problems, and recall some fundamental theoretical results. For a more gentle introduction, we refer the reader to the reinforcement learning book of Sutton and Barto [2].

Next, we take a closer look at the particular constructive scheduling MDP that we defined in Section [insert] and argue that the class of horizon policies considered there is general enough, in the sense that it always contains an optimal policy.

In Section 3, we present the policy gradient theorem and show how it leads to the well-known REINFORCE algorithm with baselines. We examine the difference between the single-step update and the episode update.

1 Broad overview of the MDP framework

We consider some set T of decision epochs, which is assumed to be a finite or countable set.[1] In general, we can distinguish between *finite-horizon* problems, in which $T = \{1, 2, \dots, N\}$ for some integer $N < \infty$ and *infinite-horizon* problems, where $T = \{1, 2, \dots\}$. At each decision epoch $t \in T$, the system or *environment* is in some state s from the state space $\mathcal{S}$ and the decision maker or *agent* needs to select some action a from the set of *admissible* actions $\mathcal{A}(s)$ in this state. We assume that $\mathcal{S}$ and $\mathcal{A}(s)$, for each $s \in \mathcal{S}$, are finite or countable sets. We assume that the system starts in some initial state s_0, according to the initial state distribution[2] $\rho_0 \in \Delta(\mathcal{S})$. Upon selecting action a in state s at decision epoch t, the system transitions to some next state s', according to the transition probability $p_t(s, a) \in \Delta(\mathcal{S})$, and the agent receives a reward $r_t(s, a, s') \in \mathbb{R}$. When dealing with finite-horizon problems ($N < \infty$), the last decision is made at epoch $t = N - 1$ and in that case, the final state s of the system at epoch N determines the so-called final reward $r_N(s)$. We will assume that $p_t \equiv p$ for all $t \in T$, which means that the system dynamics do not change over time, and similarly that $r_t \equiv r$ for $t \in T$. The Markov decision process (MDP) is then defined as the tuple

$$\mathcal{M} := (T, \rho_0, \mathcal{S}, \mathcal{A}, p, r). \tag{1}$$

[Change the final reward definition in the single intersection MDP to reflect assumption above.]

[1] When decisions are made continuously, and if we are not willing to introduce some discretization scheme to justify this assumption, the decision problem is better approached using optimal control theory.

[2] Given some countable set $A = (a_1, a_2, \dots)$, we define the *probability simplex* over A to be the set of mappings

$$\Delta(A) := \left\{ p : A \to [0, 1] \mid \sum_{a \in A} p(a) = 1 \right\},$$

so it represents all possible discrete probability distributions over A. For functions $f : X \to \Delta(A)$, mapping some element $x \in X$ to a particular distribution $p = f(x) \in \Delta(A)$, we use the notational convention $p(a) = f(a|x)$.

Decision rules and policies. We now discuss how the behavior of the agent can be modeled. At each decision epoch t, some *decision rule* d_t determines how the action is selected given the current state. There are two axes that determine the class of decision rules:

	Deterministic	Randomized
Markov	MD	MR
History-dependent	HD	HR

The simplest of these four is the class of MD decision rules, where every state is mapped to a single action, so we have $d_t(s_t) \in \mathcal{A}(s_t)$. Instead of the current state only, we could allow this mapping to depend on the whole state-action *history*

$$h_t = (s_0, a_1, s_1, \dots, a_{t-1}, s_{t-1}), \tag{2}$$

in which case we obtain the class of HD decision rules, which can be written as $d_t(h_t) \in \mathcal{A}(s_t)$. Next, we could allow the agent to specify a distribution over actions, which leads to the class MR decision rules of the form $d_t(s_t) \in \Delta(\mathcal{A}(s_t))$ and the most general class of HR decision rules of the form $d_t(h_t) \in \Delta(\mathcal{A}(s_t))$.

We obtain a policy π when we specify a decision rule d_t for each decision epoch $t \in T$. In the finite-horizon case, this means that we have $\pi = (d_1, d_2, \dots, d_{N-1})$. Some policy π is called *stationary* if the same decision rule $d_t = d$ is used at each decision epoch $t \in T$. In that case, we will just write $\pi(\cdot)$ instead of $d_t(\cdot)$.

For each of the four types $K \in \{\mathrm{MD}, \mathrm{HD}, \mathrm{MR}, \mathrm{HR}\}$, let Π^K denote the set of policies in which all decision rules are of type K, then we have $\Pi^{\mathrm{MD}} \subset \Pi^{\mathrm{HD}} \subset \Pi^{\mathrm{HR}}$ and $\Pi^{\mathrm{MD}} \subset \Pi^{\mathrm{MR}} \subset \Pi^{\mathrm{HR}}$, so Π^{HR} is the most general and Π^{MD} is the most restrictive. Stationary policies in Π^{MD} are called *pure* and we will use Π_{p} to denote the set of all pure policies.

Behavior of the system. After fixing the action selecting process in the MDP by choosing some policy π, we have essentially fixed the (random) behavior of the system. Let us illustrate this by showing how a single realization of the evolution of the system can be sampled. We will use the notation $a \sim \Delta(A)$ to denote a sample from a distribution. Consider a finite-horizon MDP and suppose we are given some policy $\pi \in \Pi^{\mathrm{HR}}$. We start by sampling the initial state $S_0 \sim \rho_0$. We sample the first action $A_1 \sim d_1(h_1) \equiv d_1(S_0)$ and the next state $S_1 \sim p(S_0, A_1)$, which results in a reward $R_1 = r(S_0, A_1, S_1)$. By repeating $A_t \sim d_t(h_t)$, $S_t \sim p(S_{t-1}, A_t)$ and $R_t = r(S_{t-1}, A_t, S_t)$ for the remaining decision epochs, we obtain a sequence of samples

$$\tau = (S_0, A_1, R_1, S_1, A_2, R_2, S_2, \dots, A_{N-1}, R_{N-1}, S_N, R_N), \tag{3}$$

which is called an *episode*. In the infinite-horizon case, the policy π induces some probability law $P^\pi(\tau)$ over the space of possible episodes, which are infinite sequences. From now on, we will assume that S_i, A_i, R_i are random variables and τ is the corresponding stochastic process.

Optimality criteria. In order to specify how we want the system to behave, we need to specify which law—and thus implicitly which policy—is most desirable. In the MDP framework, the central premise is that this can be done entirely in terms of the subprocess of rewards $R(\tau) = (R_1, R_2, \dots)$. This means that we can define some *optimality criterion* Ψ to map this reward process to a single real number, such that the goal of finding good policies can be concisely stated as the policy optimization problem

$$\max_{\pi \in \Pi^{\mathrm{HR}}} \Psi\left(R(\tau)\right). \tag{4}$$

When this maximum exists and is attained by some policy π^*, we call π^* an *optimal* policy.

In the finite-horizon case, the most natural and common optimality criterion considered in the literature is the *total expected reward*, which is defined as

$$\Psi(R(\tau)) = \mathbb{E}^{\pi}\left[\sum_{t\in T} R_t\right], \tag{5}$$

where we use $\mathbb{E}^{\pi}$ to emphasize that this expectation depends on the particular policy being used. We could also include higher moments to take into account the variability of the process, for example to model some kind of risk-aversion. In the context of modeling combinatorial algorithms, this can be useful to model probabilistic guarantees on the quality of the computed solution. There are various ways to define Ψ in the infinite-horizon case, the most common being *average reward* and *total discounted reward*, but we will not discuss these here, since they appear to be less relevant in the context of modeling combinatorial algorithms.

In the most common problem settings in the literature, finite-horizon or infinite-horizon problems with one of the optimality criteria just mentioned, it can be shown that an optimal policy exist that is in class Π_{MD}. In other words, we cannot expect to gain anything by looking at the history or using randomized action selection.

2 Optimal policies for the constructive scheduling MDP

For finite-horizon MDPs with the total expected reward objective, it is well known that among all optimal policies, we can select one that is deterministic and Markov (but not necessarily stationary). Let the constructive scheduling MDP that we introduced in Section [insert] be denoted by $\mathcal{M}_{\text{constr}}$. To keep notation concise, let $\Pi^*(\mathcal{M})$ denote the set of policies $\pi \in \Pi^{\text{HR}}$ that are optimal for $\mathcal{M}$ with respect to the total expected reward criterion (5). Our first goal is to show that we only have to consider pure policies, as stated by the followin lemma.

Lemma 2.1. *The constructive scheduling MDP satisfies* $\Pi^*(\mathcal{M}_{\text{constr}}) \cap \Pi_{\text{p}} \neq \varnothing$.

Furthermore, as we argued intuitively in the main text, the disjunctive graph state encoding contains some redundancy. In particular, it can be shown that there exist optimal policies that are indifferent to how the partial schedule was constructed up until the current decision epoch, and thus only take into account the horizon $\phi(s_t)$ at that time. Let Π_ϕ denote the set of so-called *horizon policies*, which can be defined as the set of all policies $\pi \in \Pi^{\text{HR}}$ for which there exists some function f such that $\pi = f \circ \phi$. Our intuitive argument is now made precise by the following sharpening of the above lemma.

Theorem 2.1. *The constructive scheduling MDP satisfies* $\Pi^*(\mathcal{M}_{\text{constr}}) \cap \Pi_{\text{p}} \cap \Pi_\phi \neq \varnothing$.

Pure policies. Note that $\mathcal{M}_{\text{constr}}$ does not immediately belong to the finite-horizon or infinite-horizon cases introduced in Section 1, because the number of decision epochs depends on the initial state. Therefore, we first apply the well-known trick of introducing so-called *terminal states* to formulate $\mathcal{M}_{\text{constr}}$ in terms of an equivalent infinite-horizon problem. For every state that is a complete disjunctive graph, we introduce a

Proposition 2.1. *Let $\mathcal{M}$ be a finite-horizon MDP, then* $\Pi^*(\mathcal{M}) \cap \Pi^{\text{MD}} \neq \varnothing$.

Proof. Our MDP definition in Section 1 satisfies case (a) of Proposition 4.4.3 in [1]. □

Horizon policies.

Theorem 2.2. *Let $\phi : \mathcal{S} \to \bar{\mathcal{S}}$ some state abstraction. Suppose that for all $s_1, s_2 \in \mathcal{S}$ such that $\phi(s_1) = \phi(s_2)$, the following holds*

1. $\mathcal{A}(s_1) = \mathcal{A}(s_2) =: A$,

2. $\phi(p(s_1, a)) = \phi(p(s_2, a))$ *for all* $a \in A$,

3. $r(s_1, a) = r(s_2, a)$ *for all* $a \in A$,

then there exists an optimal policy $\pi^* \in \Pi_{det}$ *such that* $\pi^* \in \Pi_\phi$.

Proof. Argue that we can change every optimal π^* into an abstract policy π^*_ϕ without changing the total reward. □

3 Policy gradient optimization

We introduce yet another perspective on the algorithm model that could be helpful in some contexts. Suppose we are given an MDP specified by the tuple $(\mathcal{S}, \mathcal{A}, p, r)$, and some policy π. Let s_0 denote some initial state and let τ denote some particular episode of states and actions, so it is a tuple

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_T). \tag{6}$$

The probability of this trajectory can be decomposed as follows

$$p(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi(a_t|s_t) p(s_{t+1}|s_t, a_t), \tag{7}$$

where $\rho_0(\cdot)$ denotes the distribution of the initial state.

Why consider non-deterministic policies? We showed that there exists an optimal pure policy for $\mathcal{M}_{\text{constr}}$, so it might appear strange to consider stationary randomized policies during optimization. However, the reasons for doing this is related to the reason for considering *relaxations* when solving integer programs. In the relaxed version of an integer program, the integer constraints are removed, which makes it much easier to solve. The final solution must of course satisfy the integer constraints, so we need some sort of *rounding procedure* to return to integer values. Similarly, we want to find an optimal deterministic policy, but we relax the constraint that each state maps into a single action by allowing distributions of actions instead. Since the final solution needs to be a deterministic policy, we need to specify some "rounding procedure" to return to degenerate distributions, for example, by performing greedy action selection, or by applying search procedures like beam search, or the active search procedure from [3].

4 MDP homomorphism

Definition 4.1. *A Markov decision process (MDP) is a tuple* $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r)$

Definition 4.2.

References

[1] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming.* Number v.414 in Wiley Series in Probability and Statistics. John Wiley & Sons, Inc, Hoboken, 2009.

[2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction.* Adaptive Computation and Machine Learning Series. The MIT Press, Cambridge, Massachusetts, second edition edition, 2018.

[3] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural Combinatorial Optimization with Reinforcement Learning, January 2017.